

T.C.
BAHÇEŞEHİR UNIVERSITY



FACULTY OF ENGINEERING AND NATURAL SCIENCES
DEPARTMENT OF COMPUTER ENGINEERING

CMP2003 – DATA STRUCTURES & ALGORITHMS (C++)
FALL TERM PROJECT
LOG ANALYZER

Kuzey Berk Yilmaz 2200014

Demir Eroglu 2200678

Baris Balkaya 2105157

Emir Ismail Genc 2202780

Lecturer: Tefvik Aytekin

24/12/2023

Contents

1	Introduction	4
1.1	Project Description and the Objective	4
2	General Planning	4
2.1	Determination of Necessary Additional C++ Libraries	4
3	Detailed Overview of the Source Code	5
3.1	Task 1: Creation and Implementation of a Unique Hash Table	5
3.2	Task 2: “std::unordered_map” Data Structure Solution	8
4	Observation	10
4.1	Total running time of Task 1 and Task 2 - Comparison	10
4.2	Required Disclaimer	11
5	Conclusion	11

1 Introduction

1.1 Project Description and Objective

The objective of the project is to develop an effective and sophisticated C++ console application which processes and analyzes a log file from a web server, with the aim of finding the top 10 most visited web pages and printing them as output.

The core data structure used for this task is a hash table, implemented as a class. This custom hash table is utilized to store filenames along with the respective number of visit counts to each web page. The program also employs the “std::unordered_map” data structure to achieve the same objective, for the sole purpose of offering a comparative benchmark, allowing for a comparison of performance and functionality between the custom hash table and the standard library’s implementation.

The purpose behind creating a custom hash table instead of “std::unordered_map” as the main data structure is its efficiency in handling key-value pairs, making it a suitable solution for quick access to filenames and their corresponding visit counts. The aspect which makes the use of hash tables great is that they facilitate the quick retrieval of information, even when handling data in large datasets.

The final output of the application includes a list of the top 10 most visited web pages, followed by the total elapsed time.

2 General Planning

2.1 Determination of Necessary Additional C++ Libraries

```
#include <iostream>
#include <unordered_map>
#include <fstream>
#include <string>
#include <vector>
#include <chrono>
#include <algorithm>
#include <cassert>
```

- “**unordered_map**” library was utilized to solve Task 2.
- “**vector**” library was utilized for dynamic array allocation.
- “**chrono**” library was utilized to measure the total elapsed time.
- “**cassert**” library was utilized to ensure that the log file is kept open.
- “**algorithm**” is a header under the standard library which includes the std::sort sorting algorithm implemented in the code.

3 Detailed Overview of the Source Code

3.1 Task 1: Creation and Implementation of a Custom Hash Table

Two different custom hashmap implementation solutions for the project were prepared.

Quadratic Probing: The Collision Resolution Method

The collision resolution method employed in both of the solutions is quadratic probing.

When a collision is detected during the insertion of a key-value pair, the “**quadProb**” method is invoked in the first solution to find the next available slot. The “quadProb” method calculates the new index by adding the square of the probe iteration to the original index and taking the result modulo the size of the hash table.

In the second solution, the probing is done by incrementing the index with a quadratic function “**(originalIndex + i * i) % HTSize**”, where “i” is incremented until an empty slot is found. If the table is full (after HTSize iterations), an error message is displayed.

This approach incrementally adjusts the index in a quadratic way, aiming to distribute collided elements more evenly throughout the hash table. If the calculated index is already occupied, the algorithm continues probing until it finds an empty slot. Quadratic probing helps mitigate clustering issues that can occur when using linear probing, and ensures a more even distribution of elements.

Solution 1

Data Structures

The primary data structures include a class named “**hashT**” representing the hash table. The class also includes a “**vector<pair<string, int>> HTable**”, functioning as the hash table itself. This vector of pairs stores the key-value associations [page names (as strings) and their corresponding frequency counts (as integers)]. The class features member variables such as “array_size” to denote the size of the hash table and an input file stream (logfile) to read from the “access_log” file.

- “**int array_size**” variable holds the size of the hash table.
- “**hashT**” is the constructor that initializes the hash table with a specified size and opens a file stream to “access_log.” If the size is not pre-defined in the array_size variable, the hashT constructor is ran without any input parameters and the size is accepted as the variable’s value.
- “**quadProb**” is the Quadratic probing function for collision resolution.
- “**int FirstHashIndex**” is the hash function to calculate the initial index for a given log name.
- “**string read_logs**” is the function to extract and return a substring representing the requested page from a log entry.
- “**void insert**” is the function to read logs from the file, hash the page names, and insert/update the hash table.
- “**void print**” is the function to sort the hash table based on frequency and print the results.

- **"vector< pair<string, int>> HTable"** is the Vector of pairs representing the hash table. Each pair consists of a page name and its frequency.
- **"FirstHashIndex function"** calculates the initial hash index for a given log name using a simple hashing algorithm.
- **"read_logs function"** extracts and returns the requested page from a log entry.
- **"quadProb function"** implements quadratic probing for collision resolution.
- **"insert function"** reads logs from the file, hashes page names, and inserts/updates the hash table using quadratic probing for collision resolution.
- **"print function"** sorts the hash table based on frequency in descending order.

Hash Function

The function calculates the initial hash index for a given string **"logName"**. It uses a hash function to iterate through each character of the string, multiplying the current hash value by 13 (since a prime number is required), and adding the ASCII value of the character. The modulo of the result **"HTable.size()"** is taken to make sure it stays within the boundaries of the hash table.

Quadratic probing is implemented by the **"quadProb"** function which takes the initial hash index **"indexX"** and the current probing iteration **"probIteration"**. By adding the square of the probing iteration to the initial index and taking the result modulo **"HTable.size()"**, a new index is calculated. This is done to find an open slot in the hash table for resolving collisions.

The insert function reads lines from the access log and extracts the page name using the **"read_logs"** function. After, it uses the initial hash index obtained from **"FirstHashIndex"**. If the calculated index is already occupied, quadratic probing is performed until an empty slot is found. Then the page name is inserted into the hash table, the count is incremented and the process is continued until the hash table is filled or the log file is exhausted.

Printing and the Sorting Algorithm

The sorting algorithm **"std::sort"** is an implementation of the introsort algorithm, which is a hybrid sorting algorithm that combines quicksort, heapsort, and insertion sort. In the **"print"** method, the **"std::sort"** function is applied to the **"HTable"** vector, which contains pairs of strings (page names) and integers (counts). The sorting is performed in descending order based on the counts, as specified by the lambda function passed to **"std::sort"**. This ensures that the pages with higher counts come first in the sorted vector.

The time complexity of **"std::sort"** is **O(N log N)** on the worst case, where N is the number of elements to be sorted.

Solution 2

Data Structures

Again, a hash table data structure is implemented using a class called **"hashT"**. The hash table is designed to store pairs of page names and their visit counts. The hash table employs an array of **"HashItem"** structures as its underlying storage:

- **struct HashItem** is a nested structure representing an item in the hash table. It contains three members:

- **string pageName**: The key, representing the page name.
- **int count**: The value, representing the count of visits.
- **int status**: Indicates the status of the hash item (0 = empty, 1 = occupied, -1 = deleted).

- **HashItem* HTable** is a pointer to an array of HashItem structures, representing the hash table itself. Memory for this array is dynamically allocated in the constructor and deallocated in the destructor.

- **int HTSize** is an integer representing the size of the hash table.

- **hashT(int size = 30013)** is a public Constructor that initializes the hash table with a default size of 30013 (a prime number for better distribution). This allocates memory for the hash table and initializes its items.

~**hashT()** is the public Destructor that deallocates the dynamically allocated memory for the hash table.

- **void insert(const string& pageName)** is a public function to insert a page name into the hash table using quadratic probing in case of collisions.

- **void retrieve(int hashIndex, string& pageName, int& count)** const function to retrieve the count of a page name at a given hash index.

- **int getHTSize() const** is a public function to get the size of the hash table.

- **int hashFunc(const string& key) const** is a private hash function to compute the index for a given key using a simple algorithm that involves iterating over characters in the key.

Hash Function

At first, the function starts by initializing an unsigned long variable **"hashVal"** to 0. This variable will store the cumulative hash value for the given key. Secondly, calculation of the hash value is being done by iterating over each character of the input string, and for each character 'ch', a running hash value 'hashVal' is updated, using the formula $\text{hashVal} = 37 * \text{hashVal} + \text{ch}$. This mathematical operation on each character ensures that the resulting hash value is influenced by the entire content of the string.

Third step is the index calculation: after processing all characters in the key, the function calculates the final index within the hash table by taking the modulus with the size of the hash table (HTSize).

This step ensures that the resulting index falls within the valid range of the hash table.

Sorting Algorithm

After populating the hash table, the program retrieves the data from the hash table and stores it in a vector named "visitVector". This vector is then sorted in descending order with a command in the executable file. Based on the visit counts using the "**std::sort**" function with a lambda function as the comparator, just like in the first solution. The time complexity is again **$O(N \log N)$** on the worst case, where N is the number of elements to be sorted.

3.2 Task 2: "std::unordered_map" Data Structure Solution

Two different "std::unordered_map" solutions for the project were prepared.

The first solution is effective overall and has **$O(n)$** time complexity, but after a period of time, the second solution which has faster execution time but a worse time complexity of **$O(n \log n)$** has been came up with.

The reason why the second solution works faster with worse time complexity compared to the first one, is because of the number of unique elements in the access log file. If the complexities were separated with unique numbers (r) in mind, the first solution has a time complexity of $O(n)$ and the second solution has a time complexity of **$O(n) + O(r \log r)$** , thus meant that the first solution would have worked faster in comparison only if a lot of unique numbers existed. But clearly this isn't the case with the given example file of the access log, so it's been decided to use the second std::unordered_map implementation for this project.

Solution 1

The **RSN** function (Read Site Names) is defined to extract the site name from each log entry, specifically from lines containing "GET" requests.

The main function begins by opening the access log file and initializing necessary variables, including two unordered maps (**STK and KTS**) for tracking site visit counts and ordering them by count. The first map, STK, stores the count of visits for each site name, while the second map, KTS, is used to organize site names based on their visit counts.

The first while loop reads each line of the log file, extracts the site name using the "RSN" function, and updates the corresponding unordered maps with visit counts and site names.

The second while loop is responsible for identifying and printing the top 10 visited sites. It iterates until either all sites are processed or the top 10 sites are identified. The loop checks if the current count (**counter**) matches the total number of distinct sites (**checkI**). If true, it prints out a message which indicates that there are fewer than 10 sites. Otherwise, it examines the next count in the unordered map (**KTS**). If the count matches the current "**tenCounter**", it iterates through the sites and prints the names along with their counts, ensuring that each site name is only printed once. The process continues until 10 sites are printed.

Lastly, the elapsed time is measured by utilizing the **C++ <chrono> library**, starting before the first while loop and ending after the second while loop. The total elapsed time is then printed at the end of the program.

Solution 2

A class called "UMap" is defined in a separate header file which is used to utilize the "unordered_map" container to analyze log data from the access log. The class has a default constructor that initializes an ifstream object, "logfile", with the log file's name.

The class includes a private ifstream object "**logfile**", a vector of pairs "**visitVector**" to facilitate sorting, and an unordered_map "visitCounts" to store the visit count for each file.

The "**read_logs**" method is for parsing log entries. A log entry string "**linex**" is taken, and by locating the position of the "GET " substring within the string, the requested file name is extracted. The method calculates the starting position as the position of "GET " plus 4 (to move past "GET "), and the ending position as the position of the next space character after the starting position. The file name is then obtained using the "**substr**" function, which extracts a substring from the log entry.

The "**to_map**" method reads log entries from the open log file, extracts file names using the "**read_logs**" method and fills an unordered_map named "**visitCounts**". The method increments the visit count associated with the extracted file name in the unordered_map for each line in the log file.

The assert statement ensures that the log file is open before processing begins.

Lastly, the "**t_print**" method facilitates the printing of the top 10 most visited pages along with their respective visit counts. The data is transferred from the "visitCounts" unordered_map to a vector named "visitVector" for sorting. The sorting is performed in descending order, based on visit counts using a lambda function and the sort algorithm. Once the vector is sorted, the method prints the top 10 entries, displaying the file names and their corresponding visit counts.

4 Observation

4.1 Total running times of Task 1 (**custom hashmap**) and Task 2 (**std::unordered_map**) (with Screenshots)

Task 1, Solution 1

```
C:\WINDOWS\system32\cmd.exe
Unique elements in the hash table: 12179
=====
Top 10 elements:
1) index.html 139961 total visits
2) 3.gif 24001 total visits
3) 2.gif 23601 total visits
4) 4.gif 8014 total visits
5) 244.gif 5148 total visits
6) 5.html 5010 total visits
7) 4097.gif 4874 total visits
8) 8870.jpg 4493 total visits
9) 6733.gif 4278 total visits
10) 8472.gif 3843 total visits

Total Elapsed Time: 0.314522Seconds
Press any key to continue . . .
```

Worst case: $O(N (\log N))$

Task 1, Solution 2

```
C:\WINDOWS\system32\cmd.exe
12179 Unique Elements in the hash table.
Top 10 most visited pages:
index.html was visited 139961 times.
3.gif was visited 24001 times.
2.gif was visited 23601 times.
4.gif was visited 8014 times.
244.gif was visited 5148 times.
5.html was visited 5010 times.
4097.gif was visited 4874 times.
8870.jpg was visited 4493 times.
6733.gif was visited 4278 times.
8472.gif was visited 3843 times.

Total Elapsed Time: 0.295079Seconds
Press any key to continue . . .
```

Worst case: $O(N (\log N))$

Task 2, Solution 1

```
C:\WINDOWS\system32\cmd.exe
index.html 139961 of total visits
3.gif 24001 of total visits
2.gif 23601 of total visits
4.gif 8014 of total visits
244.gif 5148 of total visits
5.html 5010 of total visits
4097.gif 4874 of total visits
8870.jpg 4493 of total visits
6733.gif 4278 of total visits
8472.gif 3843 of total visits

Total Elapsed Time: 0.559783Seconds
Press any key to continue . . .
```

Worst case: $O(N)$

Task 2, Solution 2

```
C:\WINDOWS\system32\cmd.exe
Top 10 most visited pages:
1) index.html Count: 139961
2) 3.gif Count: 24001
3) 2.gif Count: 23601
4) 4.gif Count: 8014
5) 244.gif Count: 5148
6) 5.html Count: 5010
7) 4097.gif Count: 4874
8) 8870.jpg Count: 4493
9) 6733.gif Count: 4278
10) 8472.gif Count: 3843

Total Elapsed Time: 0.298554 Seconds
Press any key to continue . . .
```

Worst case: $O(N (\log N))$

4.2 Required Disclaimer

The building, compilation, debugging and running process of the program has been executed on a personal computer with the specifications of:

- AMD Ryzen 7 5700U Processor,
- 16 GB RAM,
- Samsung PM991A MZALQ256HBJD-00BL2 256GB M.2 NVME SSD.

The output of “Total elapsed time” may vary depending on the strength of the hardware.

5 Conclusion

Based on the results of observations and discussions, it can be concluded that the project was a complete success. The software components worked together seamlessly and achieved the desired performance.